

KARNATAKA RESIDENTIAL EDUCATIONAL INSTITUTIONS SOCIETY

ICT Class 9 - Practical Lab Exam 1

Practical Lab Exam 1 - Answer Key

Class: Class 9

Assessment Type: Practical Lab Exam 1

Total Marks: 20

Duration: 45 minutes

Topics Covered: Python Programming and Data Structures

ICT Teaching Resources Portal

Answer Key - Grading Guidelines

Note to Teacher: This answer key provides expected answers and marking guidelines. Accept equivalent answers where appropriate. Partial marks may be awarded for incomplete but correct responses.

Lab Tasks (15 marks)

Task 1: Student Grade Manager (5 marks)

• **Answer:**

```
students = []

def add_student(name, roll_no, subjects):
    """Add a new student with marks in 5 subjects."""
    student = {
        "name": name,
        "roll_no": roll_no,
        "subjects": subjects
    }
    students.append(student)

def update_marks(roll_no, subject, marks):
    """Update marks for an existing student."""
    for student in students:
        if student["roll_no"] == roll_no:
            student["subjects"][subject] = marks
            return True
    return False

def calculate_grade(average):
    """Calculate grade based on average."""
    if average >= 90: return "A"
    elif average >= 75: return "B"
    elif average >= 60: return "C"
    elif average >= 45: return "D"
    else: return "F"

def get_average(student):
    """Calculate student's average."""
    marks = list(student["subjects"].values())
    return sum(marks) / len(marks)

def find_class_topper():
    """Find the student with highest average."""
    if not students: return None
    return max(students, key=lambda s: get_average(s))

def find_subject_toppers():
    """Find topper in each subject."""
    toppers = {}
    for subject in students[0]["subjects"]:
        best = max(students, key=lambda s: s["subjects"][subject])
```

```

        toppers[subject] = best["name"]
    return toppers

def display_report_card():
    """Display formatted report card for all students."""
    print(f"{'Name':<12} {'Roll':<6} {'Avg':<8} {'Grade':<6}")
    print("-" * 35)
    for s in students:
        avg = get_average(s)
        grade = calculate_grade(avg)
        print(f"{s['name']:<12} {s['roll_no']:<6} {avg:<8.2f} {grade:<6}")

# Test data
add_student("Amit", 101, {"Math": 85, "Science": 78, "English": 92, "Hindi": 88,
"CS": 76})
add_student("Priya", 102, {"Math": 92, "Science": 85, "English": 88, "Hindi": 95,
"CS": 90})
add_student("Rahul", 103, {"Math": 68, "Science": 72, "English": 65, "Hindi": 70,
"CS": 58})

display_report_card()
print(f"\nClass Topper: {find_class_topper()['name']}")
print(f"Subject Toppers: {find_subject_toppers()}")

```

- **Marking:**

- 1 mark for student data structure (list of dictionaries)
- 1 mark for add and update functions
- 1 mark for grade calculation logic
- 1 mark for topper and subject topper functions
- 1 mark for formatted report card display

Task 2: Inventory Management System (5 marks)

- **Answer:**

```

inventory = []

def add_item(item_id, name, quantity, price, category):
    """Add new item to inventory."""
    inventory.append({
        "item_id": item_id,
        "name": name,
        "quantity": quantity,
        "price": price,
        "category": category
    })

def update_stock(item_id, amount):
    """Increase (positive) or decrease (negative) stock."""
    for item in inventory:
        if item["item_id"] == item_id:
            item["quantity"] += amount
            return True
    return False

def search_items(query):
    """Search by name or category."""

```

```

        return [i for i in inventory if query.lower() in i["name"].lower()
                or query.lower() in i["category"].lower()]

def total_inventory_value():
    """Calculate total value of all inventory."""
    return sum(i["quantity"] * i["price"] for i in inventory)

def sort_by(field):
    """Sort inventory by price or quantity."""
    return sorted(inventory, key=lambda x: x[field])

def low_stock_alert(threshold=5):
    """Items with quantity below threshold."""
    return [i for i in inventory if i["quantity"] < threshold]

def export_to_file(filename):
    """Export inventory data to a text file."""
    with open(filename, 'w') as f:
        for item in inventory:
            f.write(f"{item['item_id']},{item['name']},{item['quantity']},{item['price']},{item['category']}\n")
        print(f"Exported to {filename}")

# Menu-driven interface
def menu():
    while True:
        print("\n=== Inventory Management ===")
        print("1. Add Item  2. Update Stock  3. Search")
        print("4. Total Value  5. Sort  6. Low Stock  7. Export  8. Exit")
        choice = input("Enter choice: ")
        if choice == "1":
            add_item(input("ID: "), input("Name: "), int(input("Qty: ")),
                    float(input("Price: ")), input("Category: "))
        elif choice == "3":
            items = search_items(input("Search: "))
            for i in items: print(i)
        elif choice == "7":
            export_to_file("inventory.txt")
        elif choice == "8":
            break

# Test
add_item("P001", "Pen", 100, 10, "Stationery")
add_item("P002", "Notebook", 50, 50, "Stationery")
add_item("L001", "Laptop", 5, 45000, "Electronics")
print("Total Value:", total_inventory_value())
print("Low Stock:", low_stock_alert())

```

• **Marking:**

- 1 mark for inventory data structure
- 1 mark for add and update stock functions
- 1 mark for search and sort functions
- 1 mark for total value and low stock alert
- 1 mark for file export function

Task 3: Pattern Generator (5 marks)

- Answer:

```
def pattern_a(rows):
    """Right-angled triangle of stars."""
    for i in range(1, rows + 1):
        print("*" * i)

def pattern_b(rows):
    """Pyramid pattern."""
    for i in range(1, rows + 1):
        print(" " * (rows - i) + "*" * (2 * i - 1))

def pattern_c(rows):
    """Number pattern."""
    for i in range(1, rows + 1):
        print(str(i) * i)

def display_pattern(choice, rows):
    """Display the chosen pattern."""
    if choice == "A":
        pattern_a(rows)
    elif choice == "B":
        pattern_b(rows)
    elif choice == "C":
        pattern_c(rows)

# Test with 5 rows
print("Pattern A:")
display_pattern("A", 5)

print("\nPattern B:")
display_pattern("B", 5)

print("\nPattern C:")
display_pattern("C", 5)
```

- Output:

Pattern A:

```
*
**
***
****
*****
```

Pattern B:

```
  *
 ***
*****
*****
*****
*****
```

Pattern C:

```
1
22
333
```

4444
55555

- **Marking:**

- 1 mark for Pattern A (right triangle)
- 1 mark for Pattern B (pyramid with spaces)
- 1 mark for Pattern C (number repetition)
- 1 mark for function structure with choice parameter
- 1 mark for custom row input support and clean code

Viva Voce (5 marks)

1. **Difference between function definition and function call.** (1 mark)

- **Answer:** A function definition declares the function with `def`, specifying parameters and body. A function call executes the function using its name followed by parentheses with arguments.

2. **Mutable vs immutable data types.** (1 mark)

- **Answer:** Mutable (list, dict, set) can be changed after creation. Immutable (int, float, string, tuple, frozenset) cannot be changed; operations create new objects.

3. **Shallow copy vs deep copy of a nested list.** (1 mark)

- **Answer:** Shallow copy creates a new outer list but inner lists are shared references. Deep copy creates completely independent copies of all nested objects. `copy.deepcopy()` is used for deep copy.

4. **Purpose of `__init__` method in a Python class.** (1 mark)

- **Answer:** `__init__` is the constructor method that initialises object attributes when an object is created. Yes, an object can be created without it using default behaviour.

5. **Linear search vs binary search.** (1 mark)

- **Answer:** Linear search checks each element sequentially ($O(n)$), works on unsorted lists. Binary search divides sorted array in half each step ($O(\log n)$), requires sorted list.